

CS3807 – Deep Learning Laboratory

Experiment 3: Implementation of Convolutional Neural Networks (CNNs) for Image Classification

B.Tech Artificial Intelligence & Data Science
Shiv Nadar University Chennai

AY 2026–27

1 Objective

The objective of this experiment is to understand how Convolutional Neural Networks work in practice by implementing the convolution operation, pooling, feature map visualization, and finally image classification using TensorFlow/Keras on the CIFAR-10 dataset.

2 Learning Outcomes

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network is made up of a convolution layer, an activation layer, a pooling layer, and a fully connected layer. The convolution operation itself is defined as

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) \quad (1)$$

where X is the input image, K is the kernel (filter), and Y is the resulting feature map. The size of the output feature map depends on the input size N , filter size F , padding P , and stride S :

$$\text{Output size} = \frac{N - F + 2P}{S} + 1 \quad (2)$$

3.1 Common Convolution Kernels

Different kernels highlight different characteristics of an image by sliding over it and combining pixel values through element-wise multiplication and summation. Table 1 summarizes the standard kernels covered in the manual.

Table 1: Summary of Common Kernels

Kernel	Purpose	Typical Application
Identity	Preserve original image	Baseline comparison
Sobel-X	Detect vertical edges	Object detection
Sobel-Y	Detect horizontal edges	Boundary detection
Laplacian	Detect edges in all directions	Medical imaging
Sharpen	Enhance fine details	Image enhancement
Box Blur	Smooth image	Noise removal
Gaussian Blur	Reduce noise, preserve structure	Computer vision
Emboss	Create 3D-like effect	Artistic filtering
Outline	Highlight object boundaries	Segmentation
Motion Blur	Simulate motion along a direction	Video processing

3.2 Numerical Example 1: Convolution

For input $X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ and kernel $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, sliding the kernel over X gives:

$$1(1) + 2(0) + 4(0) + 5(1) = 6, \quad 2(1) + 3(0) + 5(0) + 6(1) = 8,$$

$$4(1) + 5(0) + 7(0) + 8(1) = 12, \quad 5(1) + 6(0) + 8(0) + 9(1) = 14$$

which gives the feature map $\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$.

3.3 Numerical Example 2: Max Pooling

For feature map $\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$, applying 2×2 max pooling on each non-overlapping window

gives 8, 3, 6 and 9 respectively, producing $\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$.

3.4 Numerical Example 3: Parameter Calculation

For a $32 \times 32 \times 3$ input passed through a convolution layer with 16 filters of size 3×3 :

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448 \quad (3)$$

4 Dataset

The dataset used is **CIFAR-10**, consisting of small colour images across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

Table 2: Dataset Summary

Training Images	50,000
Testing Images	10,000
Classes	10
Image Size	$32 \times 32 \times 3$

5 Experimental Procedure

5.1 Task 1: Dataset Exploration

CIFAR-10 was loaded via `keras.datasets.cifar10`, its shapes were printed, ten sample images were displayed, and the class distribution was plotted.

```

1 (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data
  ()
2 print("x_train shape:", x_train.shape)
3 print("y_train shape:", y_train.shape)
4 print("x_test shape:", x_test.shape)
5 print("y_test shape:", y_test.shape)
6
7 class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
8               'dog', 'frog', 'horse', 'ship', 'truck']

```

```

x_train shape: (50000, 32, 32, 3)
y_train shape: (50000, 1)
x_test shape: (10000, 32, 32, 3)
y_test shape: (10000, 1)
Training Images: 50000
Testing Images: 10000
Image Size: 32 x 32 x 3
Number of Classes: 10

```

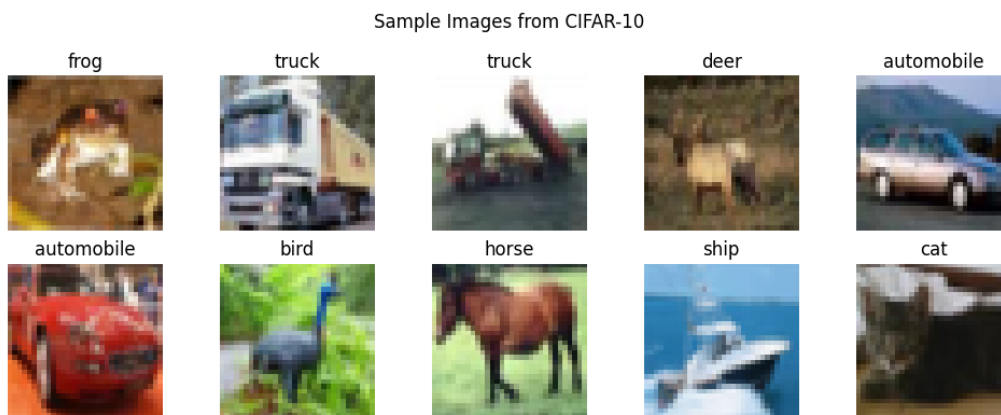


Figure 1: Sample Images from CIFAR-10

Interpretation: The samples show that CIFAR-10 images are small (32×32) colour photographs of real-world objects and animals, which is noticeably harder than a grayscale dataset like Fashion-MNIST since the network has to make sense of colour, texture and background clutter all at once.



Figure 2: Class Distribution – Training Set

Interpretation: All 10 classes have an equal number of training images (5000 each), so the dataset is perfectly balanced and plain accuracy remains a fair evaluation metric.

5.2 Task 2: Convolution with Different Kernel Sizes

A single convolution layer with 8 filters was applied to one sample image using kernel sizes 3×3 , 5×5 and 7×7 (with valid padding), and the resulting feature map shapes were recorded.

```

1 sample_img = x_train_norm[0:1]
2
3 for k in [3, 5, 7]:
4     conv_layer = layers.Conv2D(filters=8, kernel_size=k, activation='
      relu')
5     out = conv_layer(sample_img)
6     print(f"Kernel size {k}x{k} -> Feature map shape: {out.shape}")

```

Table 3: Effect of Kernel Size on Feature Map Size (32x32 input, valid padding, stride 1)

Kernel Size	Feature Map Shape
3×3	(1, 30, 30, 8)
5×5	(1, 28, 28, 8)
7×7	(1, 26, 26, 8)

As expected from the output-size formula, every increase of 2 in kernel size (with no padding) shrinks the output spatial dimensions by 2 on each side.

5.3 Task 3: Effect of Stride and Padding

The same 3×3 convolution was then tested with different stride and padding settings.

```

1 for s in [1, 2]:
2     conv_layer = layers.Conv2D(8, kernel_size=3, strides=s, padding='
      valid',
3                                     activation='relu')
4     out = conv_layer(sample_img)
5     print(f"Stride {s} -> {out.shape}")

```

```

6
7 for p in ['same', 'valid']:
8     conv_layer = layers.Conv2D(8, kernel_size=3, strides=1, padding=p,
9                               activation='relu')
10    out = conv_layer(sample_img)
11    print(f"Padding {p} -> {out.shape}")

```

Table 4: Effect of Stride and Padding (kernel = 3x3, 32x32 input)

Setting	Value	Output Shape
Stride (valid padding)	1	(1, 30, 30, 8)
Stride (valid padding)	2	(1, 15, 15, 8)
Padding (stride 1)	same	(1, 32, 32, 8)
Padding (stride 1)	valid	(1, 30, 30, 8)

Increasing the stride from 1 to 2 roughly halves the spatial dimensions of the output, while "same" padding pads the input so that the output keeps the same spatial size as the input, whereas "valid" padding uses no padding at all and therefore shrinks the output.

5.4 Task 4: Feature Map Visualization

A small one-layer model (16 filters, 3×3 , same padding) was used to visualize the feature maps produced after the first convolution layer.

```

1 feature_model = keras.Sequential([
2     layers.Conv2D(16, (3,3), activation='relu', padding='same',
3                   input_shape=(32,32,3))
4 ])
5 feature_maps = feature_model.predict(sample_img)
6 print("feature maps shape:", feature_maps.shape)
7
8 plt.figure(figsize=(12,6))
9 for i in range(8):
10     plt.subplot(2,4,i+1)
11     plt.imshow(feature_maps[0,:,:,:i], cmap='viridis')
12     plt.title(f"Filter {i+1}")
13     plt.axis('off')
14 plt.suptitle("Feature Maps after first Conv Layer")
15 plt.show()

```

feature maps shape: (1, 32, 32, 16)

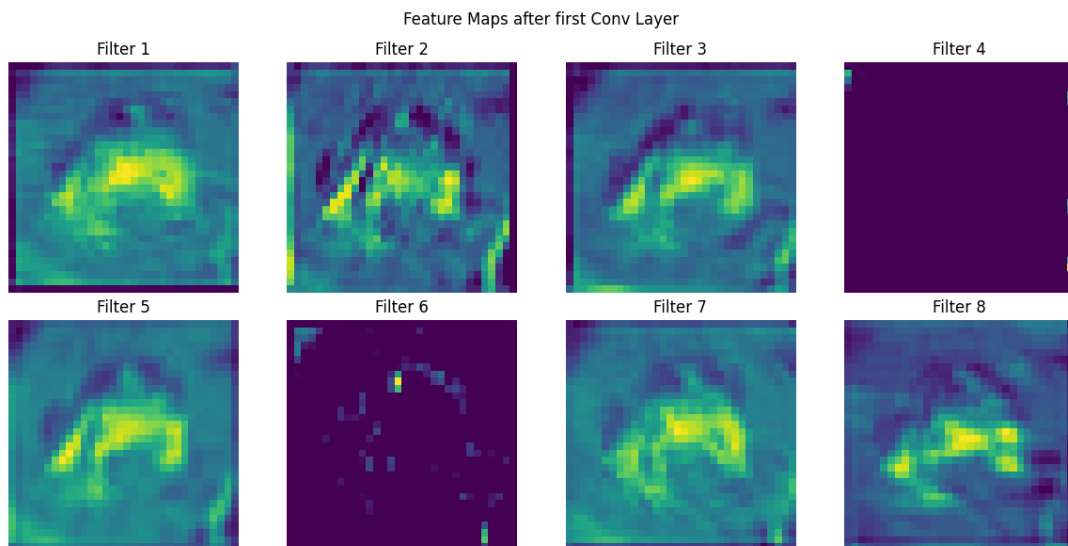


Figure 3: Feature Maps after the First Convolution Layer (8 of 16 filters shown)

Interpretation: Most of the eight filters shown clearly pick out the outline and silhouette of the object in the image (bright regions along the object's boundary), while a couple of filters (e.g. Filter 4 and Filter 6) stay almost entirely dark, meaning that particular random initialization happened to respond weakly to this specific image – this is a completely normal thing to see before the network has been trained.

5.5 Task 5: Max Pooling vs Average Pooling

Max pooling and average pooling (2×2) were applied to the 16-channel feature maps from Task 4 and compared visually and in terms of output shape.

```

1 maxpool_layer = layers.MaxPooling2D((2,2))
2 avgpool_layer = layers.AveragePooling2D((2,2))
3
4 max_out = maxpool_layer(feature_maps)
5 avg_out = avgpool_layer(feature_maps)
6
7 print("Input feature map shape:", feature_maps.shape)
8 print("After Max Pooling:", max_out.shape)
9 print("After Avg Pooling:", avg_out.shape)

```

Input feature map shape: (1, 32, 32, 16)

After Max Pooling: (1, 16, 16, 16)

After Avg Pooling: (1, 16, 16, 16)

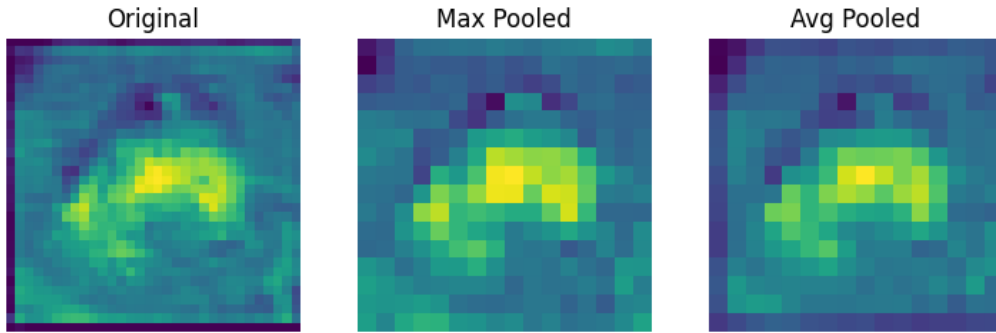


Figure 4: Original Feature Map vs Max Pooled vs Average Pooled

Interpretation: Both pooling operations halve the spatial dimensions in exactly the same way, but max pooling keeps the sharpest, most activated regions of the feature map (making the bright areas stand out more), while average pooling smooths everything out and blurs the strongest activations slightly – this is why max pooling is generally preferred when the goal is to retain the most prominent features.

5.6 Task 6: CNN Construction and Training

The final CNN follows the architecture Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax, trained with the Adam optimizer for 20 epochs with a batch size of 32.

```

1 model = keras.Sequential([
2     layers.Conv2D(32, (3,3), activation='relu', padding='same',
3         input_shape=(32,32,3)),
4     layers.MaxPooling2D((2,2)),
5     layers.Conv2D(64, (3,3), activation='relu', padding='same'),
6     layers.MaxPooling2D((2,2)),
7     layers.Flatten(),
8     layers.Dense(64, activation='relu'),
9     layers.Dense(10, activation='softmax')
10 ])
11 model.summary()
12
13 model.compile(optimizer='adam', loss='categorical_crossentropy',
14     metrics=['accuracy'])
15 history = model.fit(x_train_norm, y_train_cat, epochs=20, batch_size
16     =32,
17     validation_split=0.1)

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 64)	262208
dense_1 (Dense)	(None, 10)	650

```

=====
Total params: 282,250 (1.08 MB)
Trainable params: 282,250 (1.08 MB)
Non-trainable params: 0 (0.00 B)
-----

```

```

Epoch 1/20 - acc: 0.4775 - loss: 1.4615 - val_acc: 0.5904 - val_loss: 1.1698
Epoch 5/20 - acc: 0.7282 - loss: 0.7831 - val_acc: 0.6976 - val_loss: 0.8852
Epoch 10/20 - acc: 0.8109 - loss: 0.5383 - val_acc: 0.7018 - val_loss: 0.9601
Epoch 15/20 - acc: 0.8712 - loss: 0.3631 - val_acc: 0.6966 - val_loss: 1.1351
Epoch 20/20 - acc: 0.9170 - loss: 0.2337 - val_acc: 0.6936 - val_loss: 1.3760

```

5.7 Task 7: Model Evaluation

The trained CNN was evaluated on the 10,000-image test set with accuracy, macro precision/recall/F1, a confusion matrix, and a full classification report.

```

1 y_pred_probs = model.predict(x_test_norm)
2 y_pred = np.argmax(y_pred_probs, axis=1)
3 y_true = y_test.flatten()
4
5 acc = accuracy_score(y_true, y_pred)
6 prec = precision_score(y_true, y_pred, average='macro')
7 rec = recall_score(y_true, y_pred, average='macro')
8 f1 = f1_score(y_true, y_pred, average='macro')
9 print(classification_report(y_true, y_pred, target_names=class_names))

```

Table 5: CNN – Test Set Performance

Metric	Value
Test Accuracy	0.6799
Precision (macro)	0.6891
Recall (macro)	0.6799
F1-score (macro)	0.6834

Table 6: Per-Class Classification Report

Class	Precision	Recall	F1-score
airplane	0.69	0.72	0.71
automobile	0.81	0.77	0.79
bird	0.53	0.62	0.57
cat	0.47	0.53	0.50
deer	0.66	0.62	0.64
dog	0.56	0.60	0.58
frog	0.78	0.73	0.75
horse	0.76	0.71	0.73
ship	0.84	0.75	0.79
truck	0.79	0.76	0.77

6 Mandatory Plots

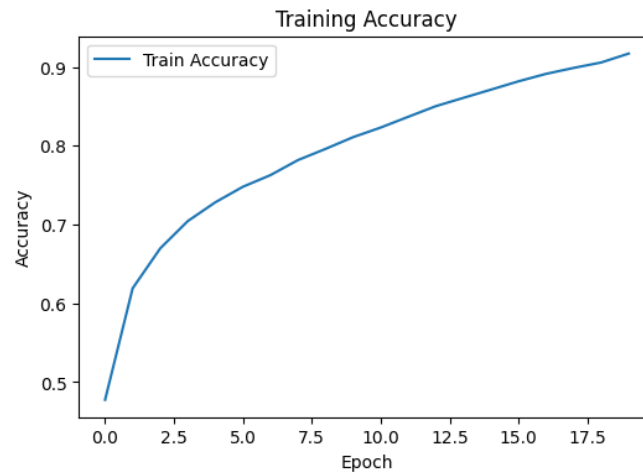


Figure 5: Training Accuracy vs Epoch

Interpretation: Training accuracy climbs almost linearly across all 20 epochs, reaching about 91.7% by the final epoch, which shows the network has more than enough capacity to fit the training data well.

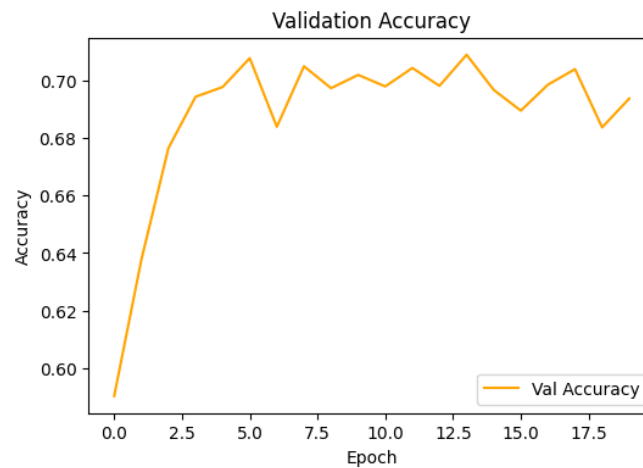


Figure 6: Validation Accuracy vs Epoch

Interpretation: Validation accuracy rises quickly in the first few epochs but plateaus around 69–71% by epoch 6 and barely improves after that, showing a large and growing gap with training accuracy – a clear sign of overfitting on this relatively small CNN once it is trained for the full 20 epochs.

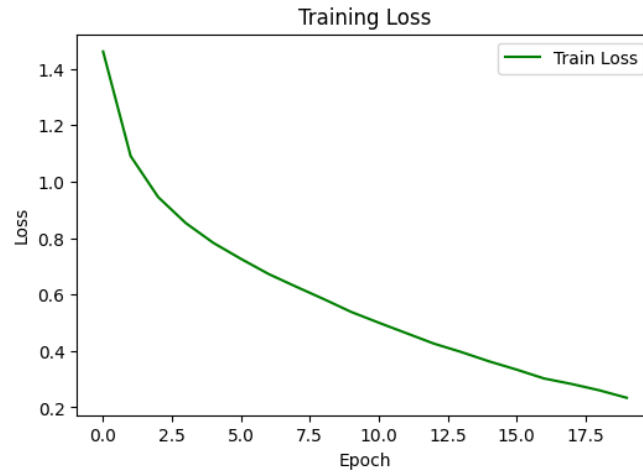


Figure 7: Training Loss vs Epoch

Interpretation: Training loss decreases smoothly and continuously throughout training, consistent with the steadily rising training accuracy seen above.

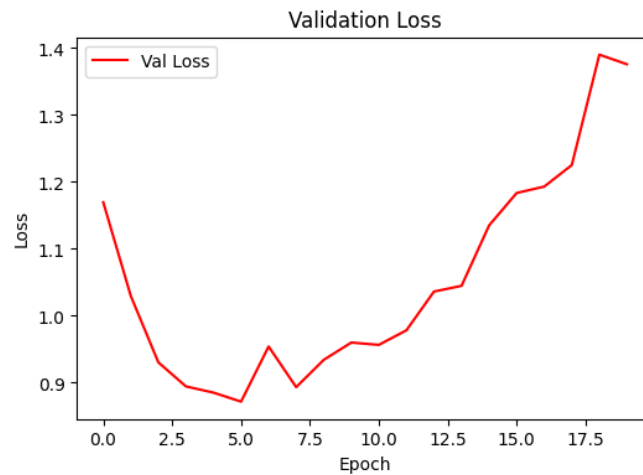


Figure 8: Validation Loss vs Epoch

Interpretation: Validation loss actually starts increasing again after around epoch 6-7, even while training loss keeps dropping – this divergence between the training and validation loss curves is the clearest evidence of overfitting in this run, and suggests that early stopping or additional regularization (e.g. dropout) would help.

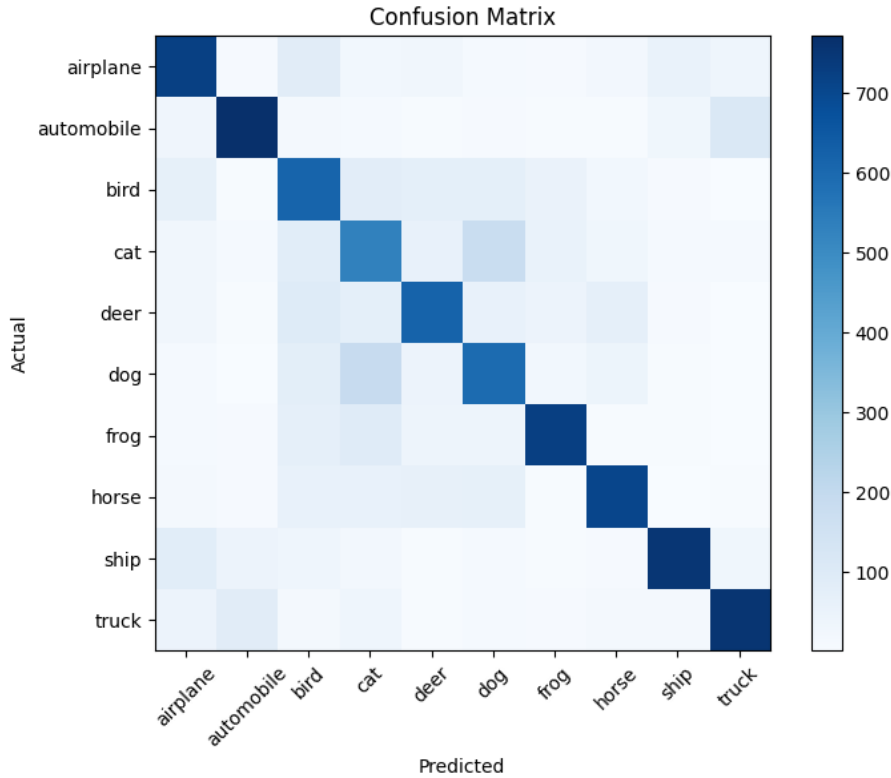


Figure 9: Confusion Matrix – CNN on CIFAR-10 Test Set

Interpretation: The model performs well on structurally distinctive classes like automobile, ship and truck, but confuses cat and dog with each other frequently, and also mixes up bird and deer to some extent – all pairs of classes that share similar overall shapes and backgrounds in low-resolution images, which matches exactly what the lower precision/recall values for cat, bird and dog in the classification report already suggested.

7 Additional Exercises

7.1 Output Size for a 64x64 Image, 5x5 Kernel, Stride 2, Padding 2

Using the output-size formula with $N = 64$, $F = 5$, $P = 2$, $S = 2$:

$$\text{Output size} = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32 \quad (4)$$

```
1 N, F, P, S = 64, 5, 2, 2
2 output = (N - F + 2*P)//S + 1
3 print("Output size:", output)
```

Output size: 32

7.2 Parameters for 64 Filters of 3x3 on an RGB Input

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792 \quad (5)$$

```
1 F, C, K = 3, 3, 64
2 params = (F*F*C + 1) * K
3 print("Trainable parameters:", params)
```

Trainable parameters: 1792

7.3 ReLU vs Sigmoid Activation Functions

ReLU outputs $\max(0, z)$, which means it has a constant gradient of 1 for any positive input and does not saturate, so networks using ReLU train faster and avoid the vanishing gradient problem that affects deep networks. Sigmoid squashes its input into the range (0,1) and saturates strongly at both ends, producing gradients very close to zero for large positive or negative inputs, which slows down or even stalls learning in deep networks. This is exactly why ReLU (and its variants) is the default choice for hidden layers in modern CNNs, with Sigmoid mostly reserved for output layers in binary classification problems.

7.4 Replacing Max Pooling with Average Pooling

The same two-convolution-layer architecture was rebuilt with `AveragePooling2D` in place of `MaxPooling2D` and trained for 5 epochs (a shorter run, sufficient for a direct comparison trend).

```
1 model_avgpool = keras.Sequential([
2     layers.Conv2D(32, (3,3), activation='relu', padding='same',
3         input_shape=(32,32,3)),
4     layers.AveragePooling2D((2,2)),
5     layers.Conv2D(64, (3,3), activation='relu', padding='same'),
6     layers.AveragePooling2D((2,2)),
7     layers.Flatten(),
8     layers.Dense(64, activation='relu'),
9     layers.Dense(10, activation='softmax')
10 ])
11 model_avgpool.compile(optimizer='adam', loss='categorical_crossentropy',
12     metrics=['accuracy'])
13 history_avg = model_avgpool.fit(x_train_norm, y_train_cat, epochs=5,
14     batch_size=32, validation_split=0.1)
```

Epoch 1/5 - acc: 0.4130 - loss: 1.6221 - val_acc: 0.5314 - val_loss: 1.3304

Epoch 5/5 - acc: 0.6692 - loss: 0.9398 - val_acc: 0.6742 - val_loss: 0.9514

At the 5-epoch mark, the average-pooling model reached a validation accuracy of 0.6742, whereas the max-pooling model (from Task 6) had already reached 0.6976 by epoch 5. This is consistent with max pooling generally preserving the strongest, most discriminative activations, which tends to help classification accuracy more than the smoothing effect of average pooling.

7.5 Increasing Filters from 16 to 64

Two lightweight single-conv-layer models (16 filters vs 64 filters, both 3 epochs) were trained and timed for a direct comparison.

```
1 model16 = keras.Sequential([
2     layers.Conv2D(16, (3,3), activation='relu', padding='same',
3         input_shape=(32,32,3)),
4     layers.MaxPooling2D((2,2)),
5     layers.Flatten(),
6     layers.Dense(10, activation='softmax')
7 ])
8 model16.compile(optimizer='adam', loss='categorical_crossentropy',
9     metrics=['accuracy'])
10 model16.fit(x_train_norm, y_train_cat, epochs=3, batch_size=32,
11     validation_split=0.1, verbose=0)
```

16 filters -> accuracy: 0.5672, time: 17.29s
64 filters -> accuracy: 0.6099, time: 18.90s

Increasing the number of filters from 16 to 64 improved test accuracy by about 4.3 percentage points ($0.5672 \rightarrow 0.6099$) for only a modest increase in training time ($17.29 \text{ s} \rightarrow 18.90 \text{ s}$), showing that a wider convolution layer captures more useful features without a proportionally large computational cost, at least at this relatively shallow depth.

8 Results

Table 7: Summary of Results

Item	Value
Final Training Accuracy (epoch 20)	0.9170
Final Testing Accuracy	0.6799
Precision (macro)	0.6891
Recall (macro)	0.6799
F1-score (macro)	0.6834
Total Trainable Parameters	282,250

9 Discussion

9.1 Why is convolution preferred over fully connected layers for images?

A fully connected layer treats every pixel as an independent input and learns a separate weight for every pixel-neuron connection, which is both extremely parameter-heavy for images and completely ignores the spatial structure of the image. Convolution instead uses small, shared kernels that slide across the whole image, which drastically reduces the number of parameters, exploits the fact that useful visual patterns (edges, textures) tend to look the same regardless of where they appear in the image (translation invariance), and naturally captures local spatial relationships between neighbouring pixels.

9.2 How does stride affect the feature map size?

As shown directly in Task 3, increasing the stride makes the kernel skip more pixels between each application, which reduces the number of positions the kernel visits and therefore shrinks the output feature map – a stride of 2 roughly halved the spatial dimensions compared to a stride of 1 in our experiment.

9.3 What is the role of padding?

Padding adds extra (typically zero-valued) pixels around the border of the input before convolution. "Same" padding is chosen specifically so that the output feature map keeps the same spatial size as the input, which both preserves information at the image borders (that would otherwise be under-represented) and makes it easier to stack many convolution layers without the feature maps shrinking too quickly. "Valid" padding uses no padding, so the output always shrinks based on the kernel size, as seen directly in Task 2 and Task 3.

9.4 Why is pooling used?

Pooling downsamples the feature maps (halving their spatial size in our 2×2 examples), which reduces the number of parameters and computation needed in later layers, provides a degree

of translation invariance (small shifts in the input don't change the pooled output much), and helps control overfitting by summarizing local regions instead of keeping every single activation value.

9.5 How do feature maps represent image characteristics?

Each feature map is the response of a single kernel applied across the entire image, so it highlights wherever that kernel's particular pattern (an edge at a certain orientation, a texture, a colour transition, and so on) is present. As seen in Figure 3, most of the filters here responded strongly to the outline of the object, effectively acting as edge detectors even before any training took place, since random convolution filters often already behave a bit like edge/gradient detectors.

9.6 Why do CNNs require fewer parameters than MLPs?

An MLP's first dense layer would need one weight for every pixel-neuron pair – for a $32 \times 32 \times 3$ image and even a modestly sized hidden layer, this quickly runs into millions of parameters. A convolution layer, by contrast, reuses the exact same small kernel (in our CNN, kernels of $3 \times 3 \times$ channels) across every spatial location in the image, so the number of parameters depends only on the kernel size and the number of filters, not on the size of the input image. This is exactly why the two convolution layers in our CNN together used only $896 + 18,496 = 19,392$ parameters, while a single dense layer mapping the same flattened $32 \times 32 \times 3$ input to just 64 units would already need over 196,000 parameters.

10 Conclusion

This experiment worked through the mechanics of a CNN from the ground up – computing convolution and pooling by hand, verifying output-size and parameter-count formulas with actual code, visualizing what a convolution layer actually "sees" in an image, and finally training a full two-convolution-layer CNN on CIFAR-10. The trained model reached about 68% test accuracy, with most of its confusion concentrated on visually and semantically similar classes such as cat and dog, or bird and deer, and it showed clear signs of overfitting once trained for the full 20 epochs, as seen in the diverging training/validation curves. The additional exercises reinforced practical, hands-on intuition for how kernel size, stride, padding, pooling type, and filter count each affect the network's output size, accuracy and computational cost – all of which are foundational choices that carry over directly into designing larger, deeper CNN architectures.

11 References

- [1] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [3] S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
- [4] TensorFlow Documentation, https://www.tensorflow.org/api_docs.
- [5] CIFAR-10 Dataset Documentation, <https://www.cs.toronto.edu/~kriz/cifar.html>.